

Conclusion

SQL 1.11 - Common Table Expressions (CTEs)

Overview

SQL 1.11 introduces one of the most impactful readability improvements available to SQL developers: **Common Table Expressions (CTEs)**.

The key idea is that a CTE is primarily a **readability and maintainability tool**, not a performance optimization tool. In most databases, a CTE is treated similarly to an equivalent nested subquery by the optimizer.

A CTE transforms complex queries into a sequence of named steps that can be read from top to bottom, making them significantly easier to understand, debug, and maintain.

Why Use CTEs?

Instead of navigating deeply nested subqueries, CTEs allow developers to break complex logic into smaller, clearly named components.

Benefits include:

- * Improved readability
- * Easier debugging
- * Better maintainability
- * Logical separation of query steps
- * Simplified collaboration among team members

Important Rules

RULE 1: CTES CAN FORM DEPENDENCY GRAPHS

A query can contain multiple CTEs.

- * Any CTE can reference any earlier CTE.
- * A CTE is not limited to referencing only the immediately preceding CTE.
- * Think of a CTE chain as a dependency graph rather than a strict pipeline.

RULE 2: CTES WORK WITH DATA-MODIFYING STATEMENTS

The **WITH** clause is not limited to **SELECT** .

It can also be used with:

- * **INSERT**
- * **UPDATE**
- * **DELETE**

A common production pattern is:

1. Use a CTE to identify rows affected by complex logic.
2. Reference that CTE in a simple data-modification statement.

RULE 3: KNOW WHEN NOT TO USE A CTE

Use a **temporary table** when:

- * Results must persist across multiple statements.
- * Indexes are required.
- * Intermediate data is reused repeatedly.

Use PostgreSQL's **MATERIALIZED** option when:

- * An expensive CTE is referenced multiple times.
- * You need to guarantee that it is computed only once.

Key Takeaways

- * CTEs improve readability and maintainability.
- * They generally do not improve performance by themselves.
- * Multiple CTEs can reference earlier CTEs.
- * They work with **SELECT** , **INSERT** , **UPDATE** , and **DELETE** .
- * Temporary tables and materialized CTEs solve different problems.

Next Topic

SQL 1.12 - Recursive CTEs

Recursive CTEs extend the power of the **WITH** clause by allowing a CTE to reference itself, enabling solutions for hierarchies, trees, and graph-like structures.

SQL 1.12 - Recursive CTEs

Overview

Recursive CTEs solve a class of problems that ordinary joins cannot handle efficiently: traversing structures whose depth is unknown.

Examples include:

- * Employee-manager hierarchies
- * Organizational charts
- * Category trees
- * Folder structures
- * Bill of materials (BOM)
- * Dependency graphs

Mental Model

A recursive CTE consists of two parts:

ANCHOR MEMBER

The anchor member executes exactly once.

Its purpose is to establish the starting rows of the recursion.

RECURSIVE MEMBER

The recursive member executes repeatedly.

Each iteration works only on the rows produced by the immediately preceding iteration.

It does **not** operate on:

- * The original anchor rows
- * The complete accumulated result set

TERMINATION CONDITION

Recursion automatically stops when an iteration produces zero new rows.

The final result is the combination of all iterations.

Production Rules

RULE 1: TRAVERSING UP AND DOWN USES THE SAME STRUCTURE

The overall pattern remains identical:

- * Anchor Member
- * **UNION ALL**
- * Recursive Member

The only difference is the direction of the join.

Traversing Down

- * Parent → Child

Traversing Up

- * Child → Parent

RULE 2: ALWAYS PROTECT AGAINST CYCLES

Never deploy a recursive CTE without a safety mechanism.

Depth Guard

Limit recursion depth using a condition such as:

```
WHERE level < 100
```

Benefits:

- * Simple
- * Fast
- * Effective protection against runaway recursion

Visited Path Tracking

Track previously visited nodes.

Benefits:

- * Precise cycle detection
- * Easier debugging

Trade-off:

- * More expensive computationally

RULE 3: PRECOMPUTE FREQUENTLY QUERIED HIERARCHIES

If a hierarchy is queried much more frequently than it changes:

Use:

- * Closure tables
- * Materialized paths

Instead of performing recursive traversal for every request.

This converts expensive recursive work into simple indexed lookups.

Key Takeaways

- * Recursive CTEs handle unknown-depth traversals.
- * Every recursive query contains an anchor member and recursive member.
- * Recursion stops automatically when no new rows are produced.
- * Always include cycle protection.
- * Precomputed hierarchies can improve performance dramatically.

Next Topic

SQL 1.13 - Ranking Window Functions

After learning how to organize logic with CTEs and traverse hierarchies with recursive CTEs, the next step is performing analytical calculations across related rows.

SQL 1.13 - Ranking Window Functions

Overview

Window functions introduce a capability that traditional aggregation cannot provide:

> Perform calculations across related rows without collapsing those rows into groups.

Unlike **GROUP BY**, window functions preserve every row while still allowing calculations across partitions of data.

All window functions operate inside an **OVER()** clause.

Understanding OVER()

The **OVER()** clause controls how the window function behaves.

PARTITION BY

PARTITION BY divides data into groups.

Calculations restart for each partition.

Examples:

- * Employees by department
- * Orders by customer
- * Transactions by account

Importantly, individual rows are preserved.

ORDER BY

ORDER BY defines the processing sequence within each partition.

This ordering is essential for:

- * Ranking
- * Running totals
- * Previous/next row calculations

Ranking Functions

ROW_NUMBER()

Assigns a unique sequential number to each row.

Characteristics:

- * No ties allowed
- * Every row receives a unique value

Common uses:

- * Pagination
- * Selecting one row per group
- * Deduplication

RANK()

Allows tied values.

When ties occur:

- * Equal values receive the same rank.
- * Subsequent ranks are skipped.

Example:

- * Rank 1
- * Rank 1
- * Rank 3

Use when ranking positions must reflect ties accurately.

DENSE_RANK()

Allows tied values without skipping numbers.

Example:

- * Rank 1
- * Rank 1
- * Rank 2

This is often the best choice when business questions focus on distinct value levels.

Examples:

- * Second-highest salary
- * Top-performing products
- * Revenue tiers

Choosing the Right Ranking Function

USE ROW_NUMBER() WHEN

- * Every row must be unique.
- * Pagination is required.
- * Duplicate ranks are undesirable.

USE RANK() WHEN

- * Ties matter.
- * Gaps after ties are acceptable.

USE DENSE_RANK() WHEN

- * Ties matter.
- * Consecutive ranking values are desired.

Window Functions and WHERE

Window functions are evaluated after the **WHERE** clause.

As a result:

- * A window function cannot be referenced directly inside the same query's **WHERE** clause.

The standard solution is:

3. Compute the window function inside a CTE.

4. Filter the result in an outer query.

This pattern is common throughout analytical SQL.

Key Takeaways

- * Window functions preserve individual rows.
- * **PARTITION BY** creates logical groups.
- * **ORDER BY** defines processing order.
- * **ROW_NUMBER()** , **RANK()** , and **DENSE_RANK()** differ primarily in how they handle ties.
- * CTEs are often used when filtering window-function results.

Next Topic

SQL 1.14 - LAG, LEAD, and Window Frames

Ranking functions answer "Where am I?"

The next family of window functions answers:

- * "What came before me?"
- * "What comes after me?"
- * "What is the running total so far?"

SQL 1.14 - LAG, LEAD, and Window Frames

Overview

SQL 1.14 completes the analytical window-function toolkit.

The major concepts include:

- * **LAG()**

- * **LEAD()**

- * Running totals

- * Moving averages

- * Window frames

These functions are heavily used in dashboards, reporting systems, and business analytics.

LAG()

LAG() retrieves a value from a previous row within the same partition.

Typical use cases:

- * Comparing current and previous sales

- * Growth calculations

- * Trend analysis

- * Month-over-month comparisons

LEAD()

LEAD() retrieves a value from a future row within the same partition.

Typical use cases:

- * Looking ahead to future events

- * Detecting the last row in a group

- * Time-series analysis

A **NULL** returned by **LEAD()** is often useful because it naturally identifies the final row in a partition.

Understanding Window Frames

Window frames determine which rows participate in a windowed aggregate calculation.

They become important whenever **ORDER BY** appears inside the **OVER()** clause.

DEFAULT FRAME BEHAVIOR

When an aggregate window function includes **ORDER BY**, the default frame behaves like a running total.

This causes calculations to accumulate from the beginning of the partition up to the current row.

NO ORDER BY

When **ORDER BY** is omitted:

- * The aggregate is calculated across the entire partition.
- * The same value appears on every row.

This is a common source of confusion for beginners.

Production Rules

RULE 1: AVOID OFF-BY-ONE ERRORS

For a true N-row moving window:

Use:

- * **(N - 1) PRECEDING**
- * **CURRENT ROW**

Examples:

- * 3-row moving average → 2 preceding rows + current row
- * 7-row moving average → 6 preceding rows + current row

Always verify window boundaries carefully.

RULE 2: PREFER ROWS OVER RANGE

For most analytical workloads:

Use:

- * **ROWS**

Benefits:

- * Predictable behavior
- * Easier reasoning
- * Better suited for moving averages and running totals

Use **RANGE** only when rows sharing the same ordering value should be treated as a single logical point.

RULE 3: DETECT EXACT THRESHOLD CROSSINGS

Sometimes the goal is not merely to determine whether a cumulative value exceeds a threshold.

Instead, you need to identify the exact row where the crossing occurred.

A common technique is:

5. Compute a cumulative total including the current row.
6. Compute a cumulative total excluding the current row.
7. Compare the two values.

This allows precise threshold detection.

Use **COALESCE()** to handle boundary **NULL** values safely.

Key Takeaways

- * **LAG()** looks backward.

- * **LEAD()** looks forward.
- * Window frames control aggregate behavior.
- * **ORDER BY** enables running calculations.
- * **ROWS** is usually safer than **RANGE** .
- * Dual cumulative totals help identify exact threshold crossings.

Course Summary

You now have the foundation for modern analytical SQL:

- * Common Table Expressions (CTEs)
- * Recursive CTEs
- * Ranking Window Functions
- * LAG and LEAD
- * Running Totals
- * Window Frames

Together, these concepts power the majority of real-world reporting, dashboarding, business intelligence, and analytical SQL workloads.